

LEARNIFY AI

MASTER IMPLEMENTATION PLAYBOOK

Fix prompts · feature roadmap · AI teaching experience · cheat sheets · branding · tooling

Prepared as a working execution document. Priorities and pricing are recommendations, not commitments.

July 2026

How to use this document

This consolidates your master checklist into something executable: a priority snapshot, then ready-to-paste prompts grouped by real fix, not by every individual bullet.

- One structural finding first: about 15 of your “instant fixes” (sidebar, analytics, command center, email templates, content manager, course pages, pricing page, community hub, coaching hub, AI tools, template mastery, payment view, creator hub, character customization, cursor, settings) are the same underlying problem — mobile responsiveness — applied to different screens. Treat it as one systematic sweep with one QA pass, not fifteen separate tickets. See §2.1.
- Pricing, wallet, and payment-flow diagrams (subscription, cart, payout, refund) were already built in the Monetization Architecture document delivered earlier in this conversation — not repeated here to avoid duplication.
- Every §2 and §3 section ends with a shaded PROMPT box — copy it directly into Claude Code, Cursor, or whatever agent is driving implementation.

1. Priority snapshot

Priority	Item	Type
P0	Mobile responsiveness sweep (15 screens)	Fix
P0	Watch Demo Center (video + mobile + content)	Fix
P0	Certificate System → Credential OS redesign	Fix
P0	AI Tools — Flashcards error	Bug
P0	Community Hub — Solve Now button broken	Bug
P0	Official badge & logo compliance (Play Store / App Store)	Fix
P1	Creator & Coach Application System + pricing	Feature
P1	Course Experience feature build (Notes/Summary/Ask AI/Playground)	Feature
P1	Coaching Hub integrations (Meet/Zoom)	Feature
P1	Wallet & Withdrawals upgrade (Cashfree Payouts)	Feature
P1	Command Center date filter cleanup	Fix
P1	XP Store reward logic	Feature
P2	Documentation Center	Feature
P2	Course Listing System (catalog/filter/search)	Feature
P2	Video platform migration off YouTube	Research + Build
P2	Billing OS — future gateway integrations	Future

Priority	Item	Type
Future	Mobile apps (Android/iOS)	Future
Future	Certificate accreditation research	Future

2. Instant fixes — prompt library

2.1 Mobile responsiveness sweep

Affects: sidebar, subscription analytics, command center, email templates, content manager, course pages (notes/summary/ask AI/exercises/playground), pricing page, community hub, coaching hub, AI tools, template mastery, payment view, creator hub, character customization, cursor animation, settings. One sweep, one checklist — and this is exactly what playwright-mcp (from our earlier design-tooling discussion) is built for: it screenshots every screen across viewports and iterates until they match.

PROMPT — paste into Claude Code / Cursor

Audit and fix mobile responsiveness across the following screens at 320px, 375px, 425px, 768px, and 1024px:
 sidebar navigation, subscription analytics, admin command center, email template previews, content manager, course experience pages (notes, summary, ask AI, exercises, playground), pricing page, community hub (feed, leaderboard, challenges), coaching hub, AI tools panel, template mastery, payment/checkout view, creator hub, character customization screen, settings.

For each screen: make the sidebar collapse into an overlay/drawer below 768px, ensure tables convert to stacked cards below 425px, ensure no horizontal scroll at any breakpoint, and ensure touch targets are at least 44px. Use Playwright MCP to screenshot each screen at each breakpoint before and after, and only mark a screen done once the after-screenshot has no overflow and no clipped content.

2.2 Watch Demo Center

Current problems: video not loading, mobile layout broken, content hidden. Needs to cover: what Learnify AI is, how to use it, how to earn as a creator/coach, and separate demo tracks per role (student/creator/coach journey).

PROMPT — paste into Claude Code / Cursor

Fix the Watch Demo Center: replace the current video player with a responsive HTML5 player (poster image, captions, playback speed, resume position) that works at all breakpoints without hidden or clipped content. Then build four demo tracks, each 60-90 seconds:
 (1) What Learnify AI is, (2) Student journey - how to learn and earn

certificates, (3) Creator journey - how to publish and earn, (4) Coach journey - how to run sessions and earn. Add a CTA at the end of each track that routes to the matching apply/signup flow.

2.3 Official badges & brand logos

Two different problems living under one bullet: (1) the Google Play / App Store badges must be the official trademarked assets, not emoji or improvised icons; (2) course-category and tech-stack logos (Python, React, AWS, Supabase, etc.) need a legitimate, consistent icon source.

- Play Store / App Store badges: these are trademarked assets with specific usage rules — pull them from Google Play's official badge generator and Apple's App Store Marketing Guidelines asset kit, don't recolor or resize outside their guidelines.
- Tech-stack / course-category logos: use an open, permissively-licensed icon set built exactly for this — simple-icons (MIT-licensed SVGs, 3000+ brands) or devicon (built specifically for programming-language/tool logos). Both are standard in EdTech for representing “what this course teaches,” not implying endorsement by the brand.

PROMPT — paste into Claude Code / Cursor

Replace all emoji-based icons in the mobile app section and course category listings with real assets:

1. Install the "simple-icons" npm package (or devicon for programming languages/frameworks) and render brand logos as SVGs for every course category and tech-stack tag.
 2. Download the official Google Play badge from Google's badge generator and the official App Store badge from Apple's Marketing Guidelines page; use them unmodified, correct size, correct clear-space padding, and link to the coming-soon page.
- Do not recolor, stretch, or recreate either store badge by hand.

2.4 XP Store reward logic

Currently undefined. Needs: redemption catalog, one-time-use enforcement, expiry, and a redemption history log.

PROMPT — paste into Claude Code / Cursor

Implement XP Store redemption logic:

- Catalog: 10% course discount, 20% course discount, bonus AI credits, certificate fee credit.
- Each reward has: XP cost, expiry (30 days from redemption unless specified), and a one-time-use flag enforced server-side.
- On redemption: deduct XP atomically, insert a redemption_history row, issue a coupon/credit token tied to the user.
- Build a "Redemption History" view showing status (active / used / expired) for each reward the user has claimed.

2.5 Command Center date filter cleanup

Remove the redundant “This Month” filter; keep Last 7 / 30 / 90 Days only, and make the filter bar responsive.

PROMPT — paste into Claude Code / Cursor

In the admin Command Center, remove the "This Month" date-range filter option entirely (it overlaps with Last 30 Days and confuses the comparison logic). Keep only: Last 7 Days, Last 30 Days, Last 90 Days. Make the filter bar collapse into a dropdown below 768px instead of wrapping.

2.6 Certificate System → Credential OS

A full redesign, moving from a single feature into a proper module: Dashboard, Templates, Designer, Verification, Wallet, Analytics. This is also where the Certificate Verification Portal (flagged as a P0 missing feature in the earlier platform audit) belongs.

PROMPT — paste into Claude Code / Cursor

Rebuild the certificate feature as a "Credential OS" module with six sub-views: (1) Dashboard - issued/pending counts, (2) Templates - saved certificate designs, (3) Designer - Fabric.js canvas editor, (4) Verification - public verification portal with QR code + lookup by certificate ID, (5) Wallet - links issued certificates to the student's earned-credentials list, (6) Analytics - issuance volume, verification-page views, most-verified courses. Certificates should render server-side via Puppeteer for consistent PDF/PNG output.

2.7 Course experience feature build

Notes, Summary, Ask AI, Exercises, and Playground need to actually exist and work on mobile; the video player needs fullscreen, captions, playback speed, and resume-watching; add a Subscribe-to-creator button and fix the instructor profile card layout.

PROMPT — paste into Claude Code / Cursor

Build out the course-page tab set: Notes (user-editable, autosave), Summary (AI-generated per lesson), Ask AI (contextual Q&A scoped to the current lesson), Exercises (graded short-answer/code), Playground (sandboxed code runner where applicable). Fix the video player to support fullscreen, captions (WebVTT), playback speed control, and resume-from-last-position. Add a "Subscribe" button on the instructor card that follows the creator, and make the instructor profile card responsive at all breakpoints.

2.8 Community Hub — Solve Now fix

The “Solve Now” button on Challenges is non-functional — this was already flagged as an active bug in the earlier platform audit.

PROMPT — paste into Claude Code / Cursor

Debug and fix the "Solve Now" button on the Challenges tab of Community Hub. Trace the click handler through to the submission flow - check for a missing route, a broken modal mount, or an auth guard silently failing. Add an error boundary so a future failure shows a toast instead of a silent no-op. Verify the fix on mobile specifically, since Community Hub's mobile layout is also flagged as broken.

2.9 AI tools — fix and upgrade

Auto Flashcards currently throws “Something went wrong.” Beyond the bug fix, all AI tools (flashcards, quiz, summary, notes, resume builder, ATS checker, roadmaps, interview prep) need prompt-quality upgrades and mobile responsiveness.

PROMPT — paste into Claude Code / Cursor

1. Reproduce and fix the Auto Flashcards "Something went wrong" error
 - check the AI response parser for unhandled malformed JSON from the model, and add a retry-with-backoff plus a user-facing fallback message instead of a raw error.
2. Audit prompts for flashcards, quiz, summary, notes, resume builder, ATS checker, career roadmap, and interview prep - tighten each system prompt with explicit output format constraints and one worked example, so output quality is consistent across runs.
3. Make every AI tool panel responsive at 320-425px.

3. Feature build-out — prompt library

3.1 Creator & Coach application system

Fields: name, email, skills, experience, portfolio, LinkedIn, resume. Admin can approve, reject, or request changes; applicant sees submitted → under review → approved/rejected. Your own pricing note: both creator per-course price and coach per-session price default to a ₹49–₹499 range, and one course (creator) or one session (coach) must be free — admin reviews every submission for copyright and quality before approval.

PROMPT — paste into Claude Code / Cursor

Build the Creator & Coach application system:

- Application form: name, email, skills, experience, portfolio URL, LinkedIn, resume upload.

- Applicant dashboard: status stepper (Submitted -> Under Review -> Approved / Rejected), with admin comments visible if "changes requested."
- Admin dashboard: list all applications with filters by status, a detail view per applicant, and approve / reject / request-changes actions.
- Pricing defaults on approval: creators can price each course from ₹49 to ₹499 but must publish at least one course completely free; coaches can price sessions from ₹49 to ₹499 but must offer at least one session free. Enforce both rules server-side, not just in the UI.
- Before final approval, route every submitted course/session to an admin copyright and quality checklist - block publish until cleared.

3.2 Coaching Hub integrations

Mobile-responsive first, then Google Meet and Zoom integration for live sessions, with scheduling, messaging, roadmaps, outcomes, and cohorts as the upgrade path. A native "Learnify Meetings" system is explicitly a later-stage goal, not now.

PROMPT — paste into Claude Code / Cursor

Upgrade the Coaching Hub:

1. Fix mobile responsiveness across the booking calendar and session list.
2. Integrate Google Meet and Zoom via their respective scheduling APIs so a booked session auto-generates a join link.
3. Build scheduling (availability windows, timezone handling), in-session messaging, and a simple outcomes/roadmap tracker per coaching relationship.
4. Add cohort support - one coach, multiple students, shared session times.

Do not build a native video/meeting system yet - Meet/Zoom integration is the right scope for this phase.

3.3 Wallet & withdrawals upgrade

Creator withdrawal via Cashfree Payouts, with transaction history, invoice download/share, and invoice branding editable from the Content Manager. Coach withdrawals follow the identical flow. This extends the wallet/payout architecture already documented in the Monetization Architecture doc — same atomic-transaction rule applies.

PROMPT — paste into Claude Code / Cursor

Upgrade Wallet & Withdrawals:

- Integrate Cashfree Payouts for creator and coach withdrawal requests (same flow for both roles).
- Transaction list: clicking a completed payout opens a detail modal with amount, date, status, and a "Download Invoice" / "Share Invoice" action.
- Invoice template: pull branding (logo, colors, footer text) from a

- Content Manager-editable settings record, not hardcoded values.
- Every balance mutation from a payout must be a single atomic transaction against the wallet ledger table - no read-then-write.

3.4 Documentation Center

Four guide tracks: Student, Creator, Coach, Admin — each with its own topic set (getting started, courses, AI tools, certificates, wallet, billing, community for students; course creation, pricing, earnings, withdrawals, analytics for creators; scheduling, sessions, pricing, earnings, withdrawals for coaches; content/user/payment/certificate management for admins).

PROMPT — paste into Claude Code / Cursor

Build a Documentation Center with four sections (Student, Creator, Coach, Admin), each a searchable list of short guide articles. Student: Getting Started, Courses, AI Tools, Certificates, Wallet, Billing, Community. Creator: Course Creation, Pricing, Earnings, Withdrawals, Analytics. Coach: Scheduling, Sessions, Pricing, Earnings, Withdrawals. Admin: Content Management, User Management, Payments, Certificates. Use the existing Sanity CMS connection so non-engineers can edit articles without a deploy.

3.5 Course Listing System

Full catalog with Beginner → Intermediate → Advanced structure, category and skill-based filtering, career-path filtering, and search/recommendations.

PROMPT — paste into Claude Code / Cursor

Build the course catalog page: category filter, skill-level filter (Beginner/Intermediate/Advanced), career-path filter, and a search bar with autocomplete. Add four rails: Recommended (based on enrolled courses), Trending (enrollment velocity), New, and Featured (admin-curated). Course cards show level badge, category icon (see §7 on logo sourcing), and progress bar if already enrolled.

3.6 Video platform strategy — no YouTube

Explicit goal: no YouTube branding, subscribe buttons, ads, or recommendations bleeding into the learning experience. Candidates to evaluate under a minimal-cost, limited-storage-budget constraint: Cloudinary (already integrated), Cloudflare R2 + Stream, Bunny.net, Mux, Supabase Storage.

Given the constraint you stated — minimal cost, limited storage budget — Cloudinary (already wired in) plus Cloudflare R2 for raw storage is the most realistic near-term combination: R2 has no egress fees, which is the

cost that usually kills a video-heavy EdTech budget on other providers. Mux and dedicated Cloudflare Stream are excellent but priced for scale you likely don't have yet — revisit once video volume is real.

PROMPT — paste into Claude Code / Cursor

Build a custom video player (not YouTube embed) backed by Cloudinary for transcoding and Cloudflare R2 for storage, with:

- Progress tracking (resume position, percent watched) written to the enrollments table.
- Completion trigger tied to certificate issuance ($\geq 90\%$ watched).
- No third-party branding, subscribe buttons, ads, or recommended-video UI in the player chrome.

Evaluate Mux and Cloudflare Stream only once monthly video storage exceeds what R2 + Cloudinary can serve within budget.

4. Creator & coach plans — refined

Earlier in this conversation the recommendation was commission-only at launch with no separate creator/coach subscription tier. You've since specified Free / Pro / Enterprise tiers for both roles with concrete per-item pricing — that's a reasonable evolution once real applicants exist, and it's compatible with the commission model underneath. Here's the combined structure:

Tier	Creator	Coach
Free	1+ mandatory free course, limited analytics, standard support, 25% commission	1+ mandatory free session, basic profile, 25% commission
Pro	Unlimited courses, advanced analytics, higher visibility, priority support	Unlimited sessions, advanced scheduling, premium profile
Enterprise	Custom — institutional/bulk publishing arrangements	Custom — institutional coaching contracts

Per-item pricing (both roles): ₹49 to ₹499, creator/coach's choice within that band, with the one-free-item rule enforced before any paid listing goes live. Admin review covers both copyright and quality before publish — this is the gate that keeps the marketplace credible from day one.

5. AI tutor voice & teaching experience

This is the layer that makes Learnify feel like a person teaching you, not a chatbot answering you. Four principles, then a system prompt you can use directly.

Design principles

- Structure every explanation as: concept → real-world example → check understanding → feedback. Never end on a bare definition.
- Warm, not chatty — encouraging tone without filler enthusiasm (“Great question!” before every reply gets old fast; save it for when something is actually a great question).
- Concrete examples over abstract ones — “imagine a for-loop like a playlist on repeat” beats a textbook definition of iteration.
- Feedback is specific and actionable — name exactly what was right, what was off, and the one thing to fix next, not a generic “good try.”

Sample system prompt

PROMPT — paste into Claude Code / Cursor

You are the Learnify AI Tutor. Your job is to teach one concept at a time so it's easy to learn and easy to remember.

For every explanation:

1. State the concept in one plain sentence - no jargon in the first pass.
2. Give one real-world example a beginner already has intuition for.
3. Ask a short question to check understanding before moving on.
4. When the learner answers, give feedback that names what was correct, what was off, and the one specific thing to adjust - never just "good job" or "not quite."

Tone: warm and encouraging, like a patient tutor who's genuinely glad to help - not effusive, not robotic. Keep responses short enough to read in one breath unless the learner asks for depth. If a learner seems frustrated, slow down and simplify rather than repeating the same explanation louder.

If Flowise agents are built out per the tooling roadmap in §8 (Career Coach Agent, Resume Reviewer Agent, Interview Coach Agent), this same system prompt structure should be the shared base every agent persona builds on — consistent voice across every AI surface in the product.

6. Cheat sheet system

A downloadable, one-page, branded quick-reference per topic — the artifact a student actually keeps open in a second tab while working, and saves for revision later.

Structure (one page, always)

- Header: topic name + Learnify AI branding + course link/QR
- Key Concepts: 4–6 bullet definitions, plain language
- Quick Reference: syntax/formula table — the part people actually scan
- Common Mistakes: 2–3 things learners get wrong, stated as “watch out for —” not just a list

- Pro Tip: one sentence, the thing an expert would tell you that isn't in the textbook

Production notes

Generate through the same Puppeteer + Fabric.js pipeline already used for certificates, so the design system (violet/teal, typography, logo placement) stays consistent across every downloadable asset on the platform. One template, populated per topic — not a bespoke design each time.

PROMPT — paste into Claude Code / Cursor

Build a Cheat Sheet generator using the existing certificate rendering pipeline (Fabric.js template + Puppeteer PDF export). Template sections, in fixed order: Header (topic + logo + QR to course), Key Concepts (4-6 bullets), Quick Reference (table), Common Mistakes (2-3 bullets), Pro Tip (one line). Constrain to a single A4/Letter page - if content overflows, trim Key Concepts before shrinking font size. Auto-generate one cheat sheet per completed course module, and add a "Download Cheat Sheet" button on the module summary screen.

Want an actual sample cheat sheet built next — e.g. a “Python Basics” one-pager — to see the template in action before wiring up the generator? Happy to build that as a follow-up.

7. Brand & logo asset system

Two different asset categories, two different sourcing rules:

Course / tech-stack logos

Use simple-icons (MIT-licensed, 3000+ brand SVGs) for general brand marks, or devicon specifically for programming languages and frameworks. Both are standard practice in EdTech — representing what a course teaches (Python, React, AWS, Supabase, Figma) is a nominative reference, not an endorsement claim, as long as the mark isn't used to imply the brand is sponsoring or certifying the course.

Store badges

Google Play and the Apple App Store badges are trademarked assets with published usage guidelines — pull them from Google's official badge generator and Apple's Marketing Guidelines asset kit respectively, at their specified minimum size and clear space, unmodified. Don't recolor, don't recreate by hand, don't substitute an emoji.

8. Tooling stack — verdict

Reviewing the tiered recommendation you brought in. Worth noting up front: that source document references “Learnify AI / DreamSync / Luxury Laundry” as one ecosystem — a few of its picks (Sereni Base, Quant, FinGPT) are wellness/finance-vertical tools that don't match anything in Learnify's actual scope so far. Marked Skip below rather than assumed relevant.

Tool	Verdict for Learnify	Why
Flowise	Adopt now	Visual agent builder for Career Coach / Resume Reviewer / Interview Coach agents — faster than hand-rolling each as custom backend code
n8n	Adopt now	Automation for onboarding emails, certificate generation, payment-success workflows — pick this one, not both n8n and Windmill
Windmill	Skip	Overlaps with n8n for this use case — running two automation platforms is maintenance overhead without a distinct need
AIProxy	Evaluate later	You already get model routing/fallback from OpenRouter — only add this if OpenRouter's own usage dashboards prove insufficient for cost tracking
Open Design	Reference only	Fine to study for layout/component inspiration — not something to integrate as infrastructure
Coolify	Future	Real migration effort (you'd own TLS, scaling, CI/CD yourself) — worth it once Vercel hosting cost is an actual budget line, not before
Sereni Base / Quant / FinGPT	Skip for Learnify	Wellness and finance verticals — not part of Learnify's current scope; revisit only if that changes
Cline / Goose / Crush	Personal dev tools	Fine for your own coding workflow — not a Learnify product feature either way

9. Course catalog & launch strategy

Launch with these 12

Full Stack Development · Python · AI & Prompt Engineering · Data Science · Cyber Security · UI/UX Design · Resume Builder · Interview Preparation · Career Roadmaps · Digital Marketing · Freelancing · Personal Branding

This is the right-sized launch list — broad enough to cover the main student intents (get a tech job, get any job, start freelancing) without spreading content production too thin.

Full catalog, by domain (condensed)

Domain	Approx. topics	Examples
Tech — Programming & Web	40+	C, Python, JavaScript, React, Next.js, Node.js
Tech — Mobile & Database	20+	Flutter, React Native, SQL, Supabase, MongoDB
Tech — AI & ML	15+	Prompt Engineering, LangChain, RAG, AI Agents
Tech — Security, Cloud, Data	25+	Ethical Hacking, AWS, Docker, Data Science
Design	15	UI/UX, Figma, Motion Graphics, Blender
Career	12	Resume, ATS, LinkedIn, Interview Prep
Business & Marketing	25+	Product Management, SEO, Content Marketing

Domain	Approx. topics	Examples
Freelancing & Content Creation	15+	Upwork, YouTube Growth, Copywriting
Office, Personal Dev, Finance, Language, Health	40+	Excel, Public Speaking, Investing, English Speaking
Academic	13	BCA, BTech CSE, DSA, DBMS

The full line-item list you provided (300+ individual course topics) is better tracked as a working spreadsheet than as prose in a document like this — something you can sort by domain, mark “in production / planned / backlog,” and assign to creators. Say the word and I’ll build that as an actual Excel tracker.